static\js\pages\dashboard.js

```
document.addEventListener('DOMContentLoaded', function() {
// Initialize severity card interactions for quick filtering
initSeverityCards();

// Timeline Chart: 5-year monthly vulnerability trends
// Shows long-term patterns and seasonal variations
const timelineChartEl = document.getElementById('timelineChart');
let timelineChartInstance = null;

if (timelineChartEl) {
function drawTimelineChart() {
const ctx = timelineChartEl.getContext('2d');

// Extract data from server-injected window objects
const labels = window.timelineData.labels || [];
const data = window.timelineData.values || [];

// Destroy existing chart if it exists
if (timelineChartInstance) {
timelineChartInstance.destroy();
}

timelineChartInstance = new Chart(ctx, {
type: 'line',
data: {
labels: labels,
datasets: [{
label: 'CVEs Per Month',
data: data,
borderColor: '#63a4ff',
backgroundColor: 'rgba(99, 164, 255, 0.1)',
fill: true,
tension: 0.3,
borderWidth: 2,
pointRadius: 4,
pointHoverRadius: 6,
pointBackgroundColor: '#63a4ff'
}]
},
options: {
responsive: true,
maintainAspectRatio: false,
plugins: {
legend: { display: false },
tooltip: {
mode: 'index',
intersect: false,
callbacks: {
title: function(context) {
return 'Month: ' + context[0].label;
},
label: function(context) {
return 'CVEs: ' + context.parsed.y;
}
}
}
},
scales: {
x: {
grid: { display: false },
ticks: {
color: '#8b9bb4',
callback: function(value, index, values) {
const label = this.getLabelForValue(value);
if (!label) return null;
if (label.slice(-2) === "01") return label.slice(0, 4);
return "";
},
autoSkip: false,
maxRotation: 0,
minRotation: 0
}
```

```
},
y: {
beginAtZero: true,
grid: { display: false },
ticks: {
color: '#8b9bb4',
stepSize: 2000,
callback: function(value, index, ticks) {
return value % 500 === 0 ? value : "";
}
}
}
},
interaction: {
mode: 'nearest',
axis: 'x',
intersect: false
},
onClick: (evt, activeEls) => {
if (activeEls && activeEls.length) {
const chart = activeEls[0].element.$context.chart;
const idx = activeEls[0].index;
const label = chart.data.labels[idx];
if (label && label.length >= 7) {
const [year, month] = label.split('-');
window.location.href = '/vulnerabilities?year=' + year + '&month=' + month;
}
}
}
});
}

// Initial draw
drawTimelineChart();

// Timeline Years Filter Handler
const timelineFilter = document.getElementById('timelineYearsFilter');
if (timelineFilter) {
timelineFilter.addEventListener('change', async function() {
const years = parseInt(this.value);
console.log(`Loading ${years} year(s) of timeline data...`);

// Show loading state
timelineChartEl.style.opacity = '0.5';

try {
// Fetch new data from API
const response = await fetch(`/api/timeline-data?years=${years}`);
const result = await response.json();

if (result.success) {
// Update global timeline data
window.timelineData = result.timeline;

// Redraw chart
drawTimelineChart();

console.log(`Successfully loaded ${years} year(s) of data`);
} else {
console.error('Error loading timeline:', result.error);
alert('Error loading timeline data. Please try again.');
}
} catch (error) {
console.error('Error loading timeline:', error);
alert('Error loading timeline data. Please try again.');
} finally {
timelineChartEl.style.opacity = '1';
}
});
}
}

// Daily Trend Chart: 30-day bar chart for recent activity
if (document.getElementById('dailyTrendChart')) {
const ctxDaily = document.getElementById('dailyTrendChart').getContext('2d');
const labels = window.timelineDataDaily.labels || [];
```

```
const dataValues = window.timelineDataDaily.values || [];

const dayLabels = labels.map((label, idx) => {
return (idx % 7 === 0) ? label.slice(5, 10) : '';
});

new Chart(ctxDaily, {
type: 'bar',
data: {
labels: dayLabels,
datasets: [{
label: 'CVEs per Day (Last 30 Days)',
data: dataValues,
backgroundColor: 'rgba(99, 164, 255, 0.8)',
borderColor: 'rgba(99, 164, 255, 1)',
tension: 0.3,
fill: true,
pointRadius: 3,
pointHoverRadius: 6
}]
},
options: {
responsive: true,
maintainAspectRatio: false,
plugins: {
legend: { display: false },
tooltip: {
mode: 'index',
intersect: false,
callbacks: {
title: function(context) {
const idx = context[0].dataIndex;
return window.timelineDataDaily.labels && window.timelineDataDaily.labels[idx] ?
window.timelineDataDaily.labels[idx] : '';
}
}
}
},
scales: {
y: {
beginAtZero: true,
grid: { display: false },
ticks: { color: '#90caf9'}
},
x: {
grid: { display: false },
ticks: {
color: '#90caf9',
maxRotation: 0,
autoSkip: false,
maxTicksLimit: 10
}
}
},
interaction: {
mode: 'nearest',
axis: 'x',
intersect: false
},
onClick: (evt, activeElements) => {
if (activeElements && activeElements.length > 0) {
const idx = activeElements[0].index;
const fullDate = window.timelineDataDaily.labels[idx];
if (fullDate) {
const year = fullDate.slice(0, 4);
const month = parseInt(fullDate.slice(5, 7));
const day = parseInt(fullDate.slice(8, 10));
window.location.href = '/vulnerabilities?year=' + year + '&month=' + month + '&day=' +
day;
}
}
}
});
}

// Severity Doughnut Chart
```

```
if (document.getElementById('severityPie')) {
const ctxPie = document.getElementById('severityPie').getContext('2d');

new Chart(ctxPie, {
type: 'doughnut',
data: {
labels: ["Critical", "High", "Medium", "Low", "Unknown"],
datasets: [{
data: [
window.severityStats.CRITICAL || 0,
window.severityStats.HIGH || 0,
window.severityStats.MEDIUM || 0,
window.severityStats.LOW || 0,
window.severityStats.UNKNOWN || 0
],
backgroundColor: [
'#f55855',
'#f8a541',
'#3b8ded',
'#42d392',
'#6b7280'
],
borderWidth: 4,
borderColor: '#1a2236',
hoverOffset: 10
}]
},
options: {
cutout: '65%',
plugins: {
legend: { display: false },
tooltip: {
callbacks: {
label: function(context) {
const label = context.label;
const value = context.parsed;
const total = context.dataset.data.reduce((a, b) => a + b, 0);
const percentage = total > 0 ? ((value / total) * 100).toFixed(1) : 0;
return `${label}: ${value} (${percentage}%)`;
}
}
}
},
onClick: (evt, activeEls) => {
if (activeEls && activeEls.length) {
const chart = activeEls[0].element.$context.chart;
const idx = activeEls[0].index;
const label = chart.data.labels[idx];
if (label) {
window.location.href = '/vulnerabilities?severity=' + label.toUpperCase();
}
}
}
});
}

// Vendor Risk Analysis Radar Chart
if (document.getElementById('vendorRiskChart')) {
const ctx = document.getElementById('vendorRiskChart').getContext('2d');
let topN = 10;
let weighted = false;

function goToVulnsForCWE(idx) {
const cwe_key = window.cweSeverityData.indices[idx];
if (cwe_key) {
window.location.href = "/vulnerabilities?q=" + encodeURIComponent(cwe_key);
}
}

function getRadarData() {
let srcAll = window.cweRadarAll;
let source = srcAll.top_10 || srcAll['all'] || window.cweRadar || {};

if (topN === 5 && srcAll.top_5) source = srcAll.top_5;
else if (topN === 10 && srcAll.top_10) source = srcAll.top_10;
else if (topN === 'all' && srcAll.all) source = srcAll.all;
```

```
if (weighted && window.cweRadarWeighted && window.cweRadarWeighted.indices) {
let srcW = window.cweRadarWeighted;
let codes = srcW.indices, names = srcW.labels, values = srcW.values;

if (topN !== 'all' && values.length > topN) {
codes = codes.slice(0, topN);
names = names.slice(0, topN);
values = values.slice(0, topN);
}
return { codes, names, values };
}

let codes = source.indices || [];
let names = source.labels || [];
let values = source.values || [];

if (topN !== 'all' && values.length > topN) {
codes = codes.slice(0, topN);
names = names.slice(0, topN);
values = values.slice(0, topN);
}
return { codes, names, values };
}

// Enhanced tooltip with CWE details, definitions, and mitigations
function radarTooltip(context) {
const code = context.label;
const name = context.dataset.meta.names ?
context.dataset.meta.names[context.dataIndex] : "";
const val = context.dataset.data[context.dataIndex];

// Get 30-day activity count for this CWE
const cwe30DayCount = window.cwe30DayCounts &&
window.cwe30DayCounts[code] ?
window.cwe30DayCounts[code] : 0;

// Get CWE definition if available
let def = "";
if (window.cweRadarDescriptions && window.cweRadarDescriptions[code]) {
def = window.cweRadarDescriptions[code];
}

// Get mitigation information if available
let mitig = "";
if (window.cweMitigations && window.cweMitigations[code]) {
mitig = "Mitigation: " + window.cweMitigations[code];
}

// Return comprehensive tooltip information
return [
`CWE ${code}`,
`Name: ${name}`,
`Number of CVEs: ${val} (last 2 years)`,
`Number of CVEs in last 30 days: ${cwe30DayCount}`,
...(def ? [def] : []),
...(mitig ? [mitig] : []),
`Learn more: https://cwe.mitre.org/data/definitions/${code.replace('CWE-', '')}.html`
];
}

// Redraw radar chart with current filter settings
function drawRadar() {
const { codes, names, values } = getRadarData();
// Clear canvas and destroy existing chart
ctx.clearRect(0, 0, ctx.canvas.width, ctx.canvas.height);
if (window.radarChartObj && window.radarChartObj.destroy) {
window.radarChartObj.destroy();
}

// Create new radar chart instance
window.radarChartObj = new Chart(ctx, {
type: 'radar',
data: {
labels: codes,                              // CWE codes as labels
datasets: [{
label: 'Vulnerability Frequency',
```

```
data: values,
backgroundColor: 'rgba(99, 164, 255, 0.2)', // Semi-transparent fill
borderColor: '#63a4ff',                 // Border color
pointBackgroundColor: '#63a4ff',        // Point color
pointBorderColor: '#fff',               // Point border
pointHoverRadius: 6,                    // Hover point size
pointRadius: 4,                         // Normal point size
pointHitRadius: 21,                     // Click detection area
meta: { names: names }                  // Store names for tooltips
}]
},
options: {
responsive: true,
maintainAspectRatio: false,
plugins: {
legend: { display: false },
tooltip: {
callbacks: { label: radarTooltip } // Use custom tooltip
}
},
scales: {
r: {                                    // Radial scale configuration
angleLines: { color: 'rgba(255,255,255,0.1)' }, // Subtle angle lines
grid: { color: 'rgba(255,255,255,0.1)' },       // Subtle grid
pointLabels: {
color: '#a9adc1',              // Label color
font: { size: 13, weight: 'bold' } // Label font
},
beginAtZero: true,                 // Start scale at zero
min: 0,
ticks: { display: false }          // Hide tick marks
}
},
// Click handler for CWE-specific search
onClick: (evt, activeEls) => {
if (activeEls && activeEls.length) {
const chart = activeEls[0].element.$context.chart;
const idx = activeEls[0].index;
const code = chart.data.labels[idx];
if (code) {
// Navigate to CWE-filtered search
window.location.href = "/vulnerabilities?q=" + encodeURIComponent(code);
}
}
}
}
});
}

// Event handler for CWE count filter dropdown
document.getElementById('radarCweCount').onchange = function() {
topN = this.value === "all" ? "all" : parseInt(this.value, 10);
drawRadar(); // Redraw chart with new filter
};

// Event handler for weighted scoring toggle
document.getElementById('radarWeighted').onchange = function() {
weighted = this.checked;
drawRadar(); // Redraw chart with weighted/unweighted data
};

// Initial chart draw with default settings
drawRadar();

// Info popover system for legend explanations
const infoIcon = document.getElementById('legendInfoIcon');
const popover = document.getElementById('legendInfoPopover');
if (infoIcon && popover) {
function showPopover(){
popover.style.display = "block"; // Show explanation popover
}
function hidePopover(){
popover.style.display = "none"; // Hide explanation popover
}
// Attach multiple event types for comprehensive interaction
infoIcon.addEventListener("click", showPopover);
infoIcon.addEventListener("mouseenter", showPopover);
```

```
infoIcon.addEventListener("focus", showPopover);    // Keyboard accessibility
infoIcon.addEventListener("blur", hidePopover);     // Keyboard accessibility
infoIcon.addEventListener("mouseleave", hidePopover);
}
}

// Initialize severity card interactions for quick filtering
function initSeverityCards() {
document.querySelectorAll(".severity-card").forEach(function(card) {
// Click handler for severity-based navigation
card.addEventListener("click", function() {
var sev = card.getAttribute("data-severity");
if (sev) {
// Navigate to severity-filtered vulnerability list
window.location.href = "/vulnerabilities?severity=" + encodeURIComponent(sev);
}
});

// Mouse enter handler for visual feedback
card.addEventListener("mouseenter", function() {
card.style.boxShadow = "0 0 0 3px #63a4ff44"; // Blue glow effect
card.style.cursor = "pointer";                 // Pointer cursor
});

// Mouse leave handler to remove visual effects
card.addEventListener("mouseleave", function() {
card.style.boxShadow = "";  // Remove glow effect
card.style.cursor = "";     // Reset cursor
});
});
}
});
```